DESIGN & ENGINEERING SUMMARY

Document AI Platform

Extracting symbols from engineering PDFs into structured, searchable domain objects

Clean Architecture · DDD · FastAPI · PostgreSQL + pgvector · PyMuPDF · OpenCV · OpenCLIP · React + Konva

Postgres-backed async pipeline (no Redis/Celery) · deployed on Render · Vercel · Neon

Live demo → https://doc-editable-platform.vercel.app

Solution

The problem

Engineering teams sit on thousands of PDF diagrams — P&IDs, schematics, flowcharts. Every symbol (valves, pumps, transmitters) is locked inside a flat raster: it can't be queried, edited, validated against data, or linked to other equipment. The knowledge is trapped in pixels.

The solution

A platform that turns every symbol in a PDF into a **first-class, editable, searchable domain object** — with type, label, geometry, custom properties, immutable version history, typed relationships, and a vector embedding.

```
PDF → Extract symbols → OCR labels → Classify → Embed → Relationship graph → Editable & searchable
```

What it does

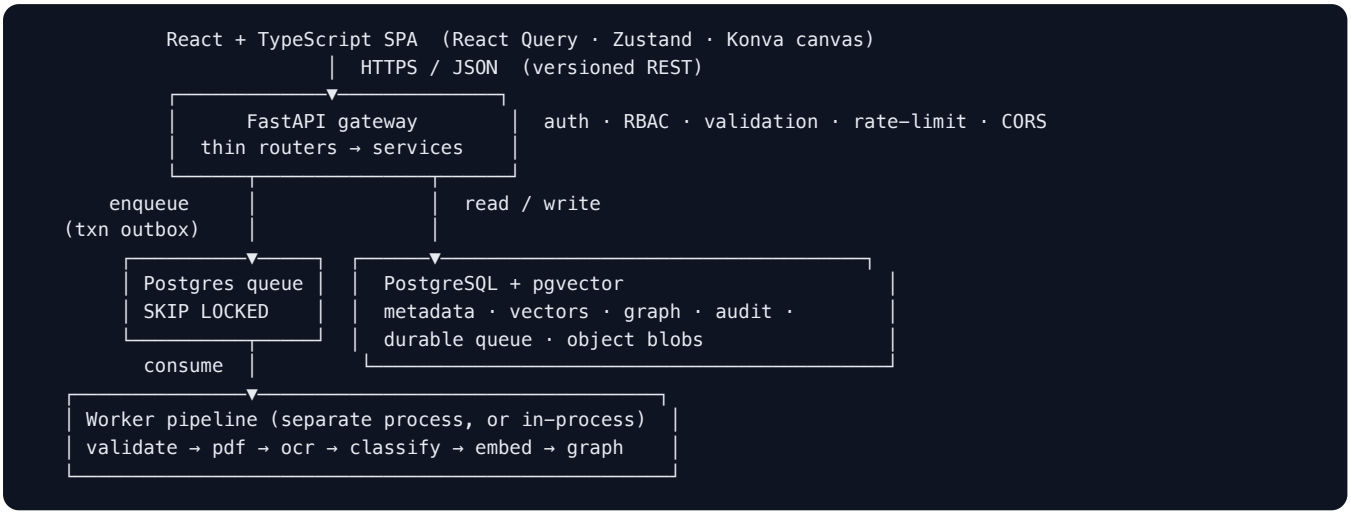
- **Ingests** PDFs with fail-closed validation (magic bytes, MIME, size + PDF-bomb caps, optional ClamAV), then processes them **asynchronously** (upload returns in <300 ms).
- **Extracts** symbols with a real CV pipeline (grayscale → threshold → denoise → contour → IoU-NMS → bbox + crop), **OCR-associates** labels, and **classifies** them (rule engine today, ML/ViT-ready behind one interface).
- **Embeds** each symbol into **pgvector** for similarity and **cross-modal** search — query by text, by a symbol, or by an uploaded crop. This is the foundation for RAG.
- **Edits** symbols on a Konva canvas (drag / resize / rotate / zoom / pan / multi-select); every change is a **versioned, audited** mutation.
- **Links** symbols into a directed, typed **relationship graph** (`pump → feeds → valve`).
- Exposes everything through a **versioned, RBAC-protected REST API** with OpenAPI docs.

A symbol is modelled as data, never a picture:

```
{ "id": "uuid", "type": "Valve", "label": "XV-200", "page": 1,
  "bbox": { "x": 120, "y": 80, "width": 40, "height": 40 }, "rotation": 0,
  "properties": { "tag": "XV-200", "pressure": "12.5" }, "embedding": [ /* 512-d vector */ ] }
```

Architecture

Built on **Clean Architecture + Domain-Driven Design**. Source dependencies point *inward only*; the domain layer is framework-free (standard library only) and 100% unit-testable.



Layer	Responsibility	Depends on
`domain`	entities, value objects, state machine, events, **ports**	nothing (stdlib)
`application`	use-case services, Unit of Work, auth/RBAC	domain
`infrastructure`	adapters: repos, queue, object store, CV/OCR/embed engines	domain (implements ports)
`interfaces`	thin FastAPI routers + DTOs, worker runtime	application
`core`	config, DI container, logging, errors, telemetry	—

Processing model. Work is async and off the request path. A document advances through an enforced state machine — `UPLOADED → VALIDATING → QUEUED → PROCESSING → OCR_RUNNING → CLASSIFYING → EMBEDDING → COMPLETED` (with `FAILED / RETRYING / CANCELLED`) — each transition timestamped and audited. The async pipeline runs on a **durable Postgres job queue** (`FOR UPDATE SKIP LOCKED`), **no Redis/Celery**. A single datastore (Neon Postgres) holds relational metadata, JSONB properties, vectors, the graph, the queue, **and** object blobs — so the whole system has one stateful dependency.

Technology stack

Area	Choice	Why this choice
API	**FastAPI** · Pydantic v2	async, first-class validation + OpenAPI
Persistence	**SQLAlchemy 2.0** (async) · Alembic	explicit Unit of Work, migrations
Database	**PostgreSQL + pgvector**	relational + JSONB + vector + graph in one store
Async pipeline	**Postgres job queue** (`SKIP LOCKED`)	durable, retryable, **no broker to operate**
PDF / CV	**PyMuPDF · OpenCV**	fast render + classic, controllable extraction
OCR	**PaddleOCR** *(optional)*	strong on rotated technical text
Embeddings	**OpenCLIP** / hash fallback	joint image–text space → cross-modal search & RAG
Object storage	**Postgres blobs** (default) or S3/MinIO/GCS/R2	single-datastore deploys, swap to S3 at scale
Security	**JWT** (access+refresh) · **Argon2** · RBAC · rate-limit · ClamAV	defense in depth
Frontend	**React 18 · TypeScript** · React Query · Zustand · **Konva**	performant 2-D symbol editing
Observability	**structlog** (JSON, correlation IDs) · **Prometheus** · OpenTelemetry	logs, metrics, traces
Testing / CI	**Pytest** (90% gate) · ruff · mypy · bandit · pip-audit · trivy · Docker	quality enforced in CI
Deploy	**Render** (API + in-process worker) · **Vercel** · **Neon**	zero paid add-ons

Why one store. Postgres + pgvector serves relational data, JSONB custom properties, vector similarity (HNSW index), the symbol graph (adjacency + recursive CTE), the job queue, and blob storage. Each boundary that might later need a specialist engine — vectors → Qdrant/Milvus, graph → Neo4j — is hidden behind a **port**, so the migration is an adapter swap, not a rewrite.

Engineering depth

The decisions that make this production-grade rather than a prototype:

- **Broker-free durable queue.** The pipeline runs on a `pipeline_tasks` table claimed with `SELECT … FOR UPDATE SKIP LOCKED`. It keeps everything a broker gives you — concurrent disjoint claims, per-stage retries with exponential backoff, dead-lettering, and crash recovery via lease reclaim — with **zero extra infrastructure**.
- **Transactional outbox.** Upload commits the `Document`, its audit row, **and** the first queue task in one transaction — so a committed document always has its task and a rolled-back upload leaves no orphan work. There is no "committed-but-lost-the-message" window.
- **Framework-free domain.** The domain imports only the standard library. The state machine, geometry (IoU, centroid distance), and label-association algorithm are pure functions — deterministically unit-tested, which is what makes the 90% coverage gate meaningful.
- **Everything is a port.** OCR, embedder, classifier, object store, and graph store are interfaces. Swapping PaddleOCR → Tesseract, hash → OpenCLIP, Postgres-graph → Neo4j, or Postgres-blobs → S3 is **one adapter, zero domain change** — and the classifier is explicitly designed to evolve rule-engine → ML → Vision Transformer behind the same interface.
- **Idempotent, independently-retryable stages.** Each stage is keyed by `(document, stage)`; re-running `pdf_extract` rebuilds from a clean slate while later stages update in place, so a retry never duplicates symbols or edges, and a failure in `embed` never re-runs `ocr`.
- **Versioned, audited edits.** Every canvas edit snapshots the prior state into an immutable `symbol_versions` row and writes an `audit_logs` entry with the actor and correlation id — a complete, queryable history of who changed what.
- **Owner-scoped cross-modal search.** A symbol, free text, or an uploaded crop all map into the **same** pgvector space, and results are scoped to the caller's documents in the repository — relevance and authorization in a single query.
- **One image, two roles.** The same container runs as a pure API, an API **with the worker in-process** (single free deploy), or a dedicated worker — chosen by one env flag, and safe to scale to N replicas because `SKIP LOCKED` hands out disjoint work.
- **Observability + correctness.** A correlation id is generated at the edge and threaded through every log line **across the async boundary** (API → queue → worker); errors are RFC-9457 `problem+json` with stable machine codes. Integration and end-to-end tests run against **real Postgres + pgvector and real PyMuPDF/OpenCV extraction**, including the retry → dead-letter path.